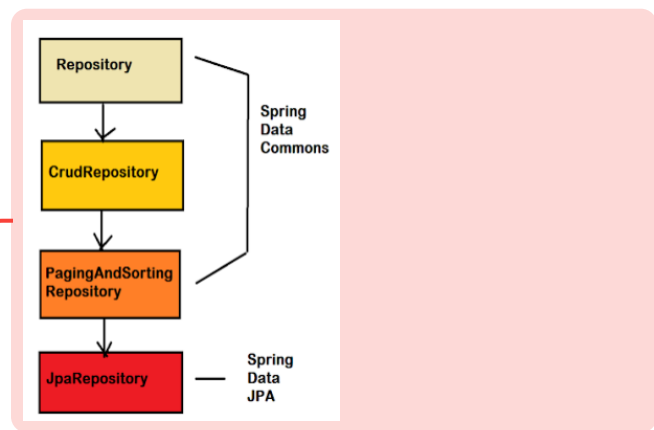


Java Web Services - Session 03

Spring Data JPA



tập hợp các interface cung cấp các phương thức làm việc giúp giảm thiểu lượng code thông thường lặp đi lặp lại ở tầng truy xuất dữ liệu .

Repository là interface gốc nắm bắt entity cần quản lý và kiểu dữ liệu ID của entity đó.

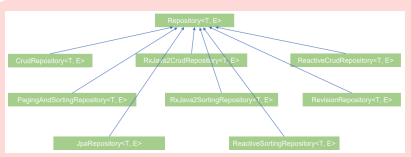
public interface Repository<T, ID>

T là kiểu dữ liệu của entity muốn quản lý

ID là kiểu dữ liệu của id của entity muốn quản lý

Repository là một marker interface : chỉ sử dụng để đánh dấu , phân loại mà không khai báo các phương thức.

Các interface kế thừa Repository



1 Interface CrudRepository

```
public interface CrudRepository<T, ID> extends Repository<T, ID> {
    T findById(ID id);
    T findOne();
    List<T> findAll();
    T save(T entity);
    void delete(T entity);
    void deleteById(ID id);
}
```

2 Interface PagingAndSortingRepository

```
public interface PagingAndSortingRepository<T, ID> extends Repository<T, ID> {
    Pageable<T> findAll(Sort sort);
    Pageable<T> findAll(Pageable pageable);
}
```

3 Interface JpaRepository

Kế thừa từ 2 interface là ListCrudRepository và ListPagingAndSortingRepository

Cung cấp thêm nhiều chức năng làm việc hơn

4 Các interface tùy biến

Có thể định nghĩa các interface để thực hiện các thao tác đặc thù

```
interface UserRepository extends JpaRepository<User, Long> {
    Page<User> findAllByLastName(String lastName);
}

interface LongRepository extends JpaRepository<Long, Long> {
    Long findById(Long id);
}
```

Phương thức truy vấn

1 Query methods

Query method là những phương thức được khai báo trong repository interface có nhiệm vụ lấy thông tin từ cơ sở dữ liệu

Query method giúp cho việc lấy thông tin từ cơ sở dữ liệu mà không cần viết một câu query nào.

```
public interface CustomerRepository extends JpaRepository<Customer, Long> {
    public Page<Customer> findAllByLastName(String lastName);
    public Page<Customer> findAllByLastNameAndFirstName(String lastName, String firstName);
}
```

2 Các bước khai báo phương thức truy vấn

Khai báo một interface kế thừa Repository hoặc các interface con của nó

Xác định kiểu của entity và kiểu của Id

Có thể không kế thừa các interface repository có sẵn bằng cách sử dụng annotation @RepositoryDefinition

Khai báo các phương thức truy vấn tùy biến (theo quy tắc đặt tên)

Cơ chế hoạt động của repository store cho phép 2 hình thức để tạo ra các câu lệnh truy vấn

1 Dựa vào tên của phương thức

```
UserRepository.findByIdAndLastName(Long id, String lastName);
```

2 Dựa vào câu lệnh truy vấn được khai báo cụ thể

```
@Query("select u from User u where u.emailAddress = ?1")
User findByIdAndEmail(String emailAddress);
```

3 Tạo câu truy vấn động

Cơ chế này duyệt qua tên của phương thức với các tiền tố như find..By, read..By, query..By , count..By và get..By để xây dựng câu truy vấn.

Có thể sử dụng thêm một số từ khóa khác, chẳng hạn như Distinct, Asc, Desc, Or, And...

```
public interface UserRepository extends JpaRepository<User, Long> {
    Page<User> findAllByLastNameAndEmail(String lastName, String email, Pageable pageable);
    Page<User> findAllByLastNameAndEmailAndFirstName(String lastName, String email, String firstName, Pageable pageable);
}
```

Sử dụng các annotation để định nghĩa câu truy vấn thông qua SQL hoặc JPQL : @Query, @Modifying, @Transactional, ...

Tạo bean repository

Có thể sử dụng XML hoặc Annotation

SpringData sẽ dò tìm và tạo ra các proxy tương ứng với từng repository để thao tác với dữ liệu

Sử dụng repository tùy biến thông qua cơ chế Injection

Tên của bean được sinh ra dựa trên tên của repository. Chẳng hạn UserRepository sẽ có một bean là userRepository

Pageable, Slice

Pageable là đối tượng hỗ trợ phân trang

- first()
- getOffset()
- getPageNumber()
- getPageSize()
- getSort()
- hasPrevious()
- isPaged()
- next()
- previousOrFirst()

Các phương thức thông dụng của Pageable:

Slice là đối tượng hỗ trợ phân trang, nhưng không biết được tổng số lượng trang (Pageable biết tổng số lượng trang)

Slice có thể tốt hơn Pageable về mặt hiệu năng khi làm việc với nhiều dữ liệu (do không phải đếm tổng số lượng trang)

Các phương thức thông dụng của Slice:

- getContent()
- getNumber()
- getPageable()
- getPageSize()
- getSort()
- hasNext()
- hasPrevious()
- isFirst()
- isLast()

Sort

Sort là đối tượng hỗ trợ sắp xếp khi truy vấn

Các constructor thông dụng:

```
Sort(Sort.Direction direction, List<String> properties)
```

Các phương thức static thông dụng

- by(List<Sort.Order> orders)
- by(Sort.Direction direction, String... properties)
- by(Sort.Order... orders)

Các phương thức thông dụng của Sort

- and(Sort)
- Ascending()
- Descending()
- Iterator()

Hạn chế số lượng kết quả

Sử dụng từ top hoặc first để hạn chế số lượng kết quả

Có thể thêm một giá trị số để quy định số lượng kết quả

Nếu không có giá trị số thì 1 kết quả sẽ được trả về